A Minor Project Proposal Report on

Flint Secure: Real-Time Fraud Detection and Device Intelligence Platform

Submitted in Partial Fulfillment of the Requirements for

the Degree of **Bachelor of Engineering in Computer Engineering**

under Pokhara University

Submitted by:

Aaditya Yadav, 231201

Apil Khadka, 231210

Ujashna Dangol, 231247

Date:

05 Jun 2026

Department of BECE

NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur, Nepal.

ABSTRACT

Nepal's digital payments are growing fast, but many payment providers still use simple fixed rules that miss organised fraud. SIM swap attacks increased sharply in recent years [1], and banks face stronger compliance checks. This project proposes **Flint Secure**: a fraud and device-intelligence service that Nepali providers can host themselves. The team will build a Go-based API and Go background workers with Redis, NATS, and PostgreSQL. Two main services are planned: a **device identification API** using a three-step fingerprint match, and a **transaction scoring API** using seven risk factors plus SIM-swap safety rules. A web SDK will collect device signals safely on the client side. The expected outcome is faster, clearer fraud decisions (ALLOW, FLAG, or BLOCK), with scoring designed to stay under 50 ms for most requests and about 3,200 requests per second in load tests. Success will be checked using k6 load scripts and roughly 168 automated test cases (at least 92% pass rate), as required for the minor project.

Keywords:

fraud detection, device fingerprinting, device intelligence, risk scoring, payment service provider, SIM swap, digital payments, transaction monitoring, Nepal, web SDK

Contents

List of Tables

List of Figures

1 Problem Statement

In the last five years, mobile wallets such as eSewa, Khalti, and IME Pay have grown quickly in Nepal. The Nepal Rastra Bank reports more than ten million mobile-banking users (2024). More users means more fraud risk. Most payment service providers (PSPs) still block fraud with simple rules (for example, a maximum amount or a daily limit) that cannot catch organised attacks.

Three main gaps motivate this project:

1. Rules are updated manually and do not adapt to new attacks such as SIM swap, stolen passwords, or fake devices.
2. Transactions are not linked to a stable device, so one phone used for many accounts (money-mule behaviour) stays hidden.
3. Systems rarely learn normal user habits (typical amounts, usual hours, usual recipients), so fraud is noticed only after money is lost.

FIU-Nepal reports a large year-on-year rise in SIM swap fraud [1]. Nepal's FATF grey-list status also pushes banks to improve monitoring. When controls are weak, fraud is missed; when rules are too strict, honest customers are blocked and providers lose business.

2 Project Objectives

2.1 Main Objective

The main objective is to design, implement, and test a real-time fraud and device-intelligence platform for Nepali Payment Service Providers (PSPs). The goal is to achieve stable device identification that persists across browser or app updates, along with risk scoring that incorporates SIM-swap detection as a key signal.

On performance, the target is for the system to respond to score requests in under 50 ms for the majority of cases (aiming for around 95%), while being able to sustain a throughput in the range of approximately 3,200 requests per second under planned load testing conditions.

To validate these goals, a comprehensive automated test suite (targeting roughly 168 test cases) will be developed, with a target pass rate of at least 92%. The system design, build steps, and validation methodology are described in Sections 5 and 6.

3 Significance of the Study

International tools such as Stripe Radar, Sift, and Sardine are strong but costly for typical Nepali wallet volumes and are not tuned for local problems like SIM swap. Flint Secure is significant because it will offer **affordable, local** fraud infrastructure using open software (Go, Redis, NATS, PostgreSQL) that a PSP can run on low-cost cloud hardware.

The project will also show that a minor-project team can meet clear performance and security targets suitable for a pilot deployment.

4 Literature Study/Review

This section reviews prior work on device fingerprinting, payment fraud, velocity checks, mobile account takeover, and the Nepali context. It supports the design in Section 5.

4.1 Device Fingerprinting and Identity Stability

Eckersley [2] showed that browser signals can identify devices without cookies. Laperdrix et al. [3] used canvas and graphics rendering to trace hardware differences; this idea will guide our first-level fingerprint in the SDK. Vastel et al. [4] studied how fingerprints change after updates; their work supports our planned three-step server match: exact hash, recovery by stored device ID, then fuzzy comparison when the fingerprint drifts. Separating device identification from transaction scoring keeps the payment path fast.

4.2 Statistical and Machine-Learning Fraud Detection

Bolton and Hand [5] reviewed statistical fraud methods and ranked velocity features (how often, how much, and to whom money moves) among the best real-time indicators. We will store these counts in Redis for quick lookup on each score request. West and Bhattacharya [6] surveyed machine-learning fraud systems; ensembles often work better, but banks also need simple, explainable rules. Fiore et al. [7] discussed handling rare fraud cases when training data is small. Dal Pozzolo et al. [8] explained how to calibrate risk scores into clear bands. For this semester we will use **rule-based scoring** with published weights; machine learning is not planned for this semester's deliverable.

4.3 Mobile Account Takeover and National Context

Mjaaland et al. [9] described SIM swap attacks on mobile banking. Combined with FIU-Nepal findings [1], this justifies strong phone-to-device checks in our scoring model. Better monitoring also supports national compliance goals, not only private profit for one company.

4.4 Commercial Platforms and Research Gap

Stripe Radar, Sift, and Sardine offer mature cloud APIs, but per-transaction pricing and foreign calibration limit their use for many Nepali PSPs. Auditors also prefer transparent rules. Flint Secure will fill this gap with **open, self-hostable** software and weights adjusted using local fraud reports [1], payment statistics, and a planned pilot data set (see Section 6).

5 Proposed Methodology

This section explains **how** the system will be built: process, tools, main steps, and design diagrams (Figures 1, 2, 3, 4, and 5).

5.1 Software Development Life Cycle

We will follow an **Agile** plan with two-week sprints (Table 2). Each sprint will end with testable code and a short supervisor meeting (NCIT Appendix B log).

5.2 Tools and Technologies

The planned stack includes Go with the chi router, two Redis instances, PostgreSQL, NATS JetStream, Go workers, a JavaScript web SDK, Docker Compose for local testing, k6 for load tests, and deployment on any VPS or cloud provider available to the team.

5.3 Repository Structure and Implementation Phases

Code will be organised in a single repository with folders for the API, workers, web SDK, and database migrations. Work will proceed in phases:

5.4 Phase 1: Infrastructure and Middleware

Step 1: Databases and messaging. Set up PostgreSQL for clients, keys, transactions, and alerts; Redis for devices and speed-sensitive counters; NATS for events such as new devices and scored payments.

Step 2: Security middleware. Each API call will pass checks for request ID, client IP, location lookup (GeoIP), API key, rate limits, and optional signed requests to stop replay attacks.

5.5 Phase 2: Device Identification API

Step 3: Web SDK. The web SDK will collect device signals, hash them on the client, and call the device identification API.

Step 4: Matching on the server. The server will try, in order: (1) exact fingerprint match; (2) match by saved device ID if the fingerprint changed slightly; (3) fuzzy match among similar devices; (4) create a new device ID if none fit, then notify background workers.

5.6 Phase 3: Risk Scoring API

Step 5: Load risk data. Before scoring, the API will read device trust, user habits, speed counters, recipient risk, and client settings from Redis in one batch.

Step 6: Calculate score. A single scoring function (no database calls inside it) will combine seven factors (device, amount, time, speed, network, identity, and recipient) and return a score from 0 to 100 with ALLOW, FLAG, or BLOCK. Hard rules will force BLOCK for clear SIM swap or datacenter abuse.

Step 7: Background updates. After the response is sent, the system will update speed counters and publish an event for workers without delaying the customer.

5.7 Phase 4: Workers and Deployment

Step 8: Background workers. Go worker services will save transactions, update user profiles, adjust device trust, and refresh settings in Redis.

Step 9: Deployment. Docker Compose for development; pilot hosting on any VPS or cloud provider available, with HTTPS and health checks.

5.8 System Design Diagrams

The following figures summarise the proposed architecture.

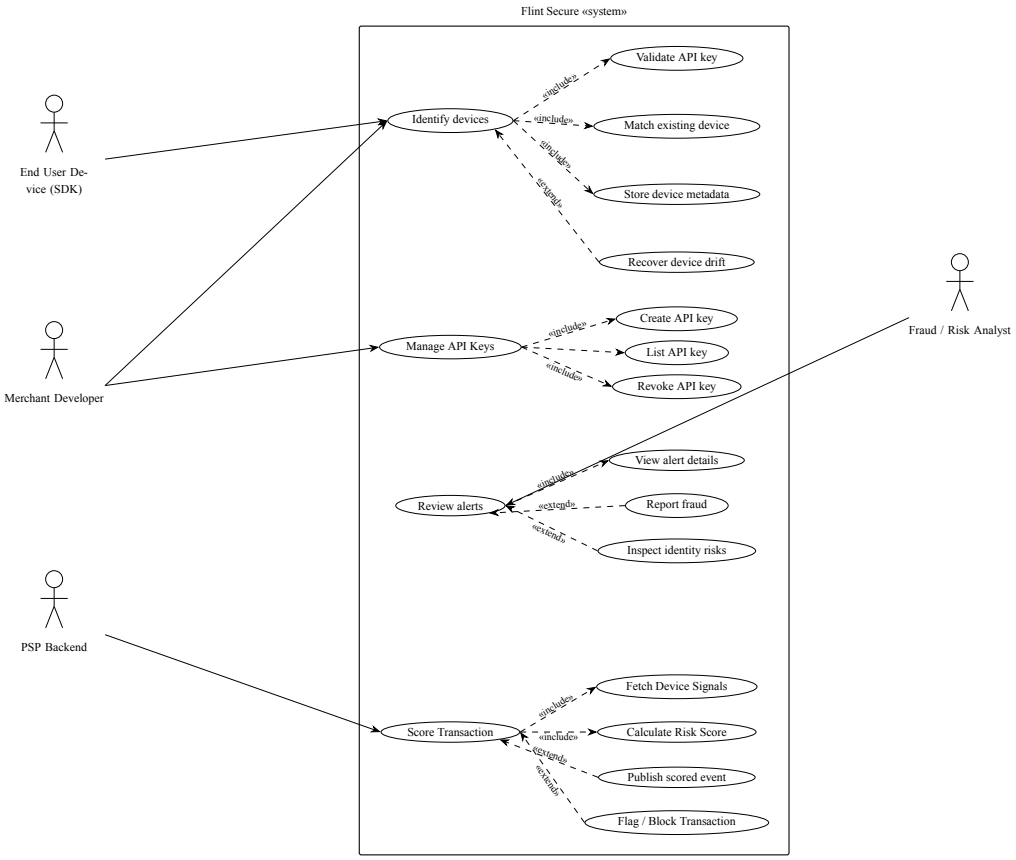

Figure 1: Use case diagram (Flint Secure system boundary).

Figure 1 defines the planned actors and their goals. The **End User Device (SDK)** and **Merchant Developer** will call *Identify devices*, which includes validating the API key, matching an existing device, and storing metadata; *Recover device drift* extends identification when fingerprints change. The merchant developer will also *Manage API Keys* (create, list, revoke). The **Fraud / Risk Analyst** will *Review alerts*, optionally *report fraud* or *inspect identity risks*. The **PSP Backend** will *Score Transaction*, including fetching device signals and calculating a risk score, then optionally publishing a scored event or flagging/blocking the payment.

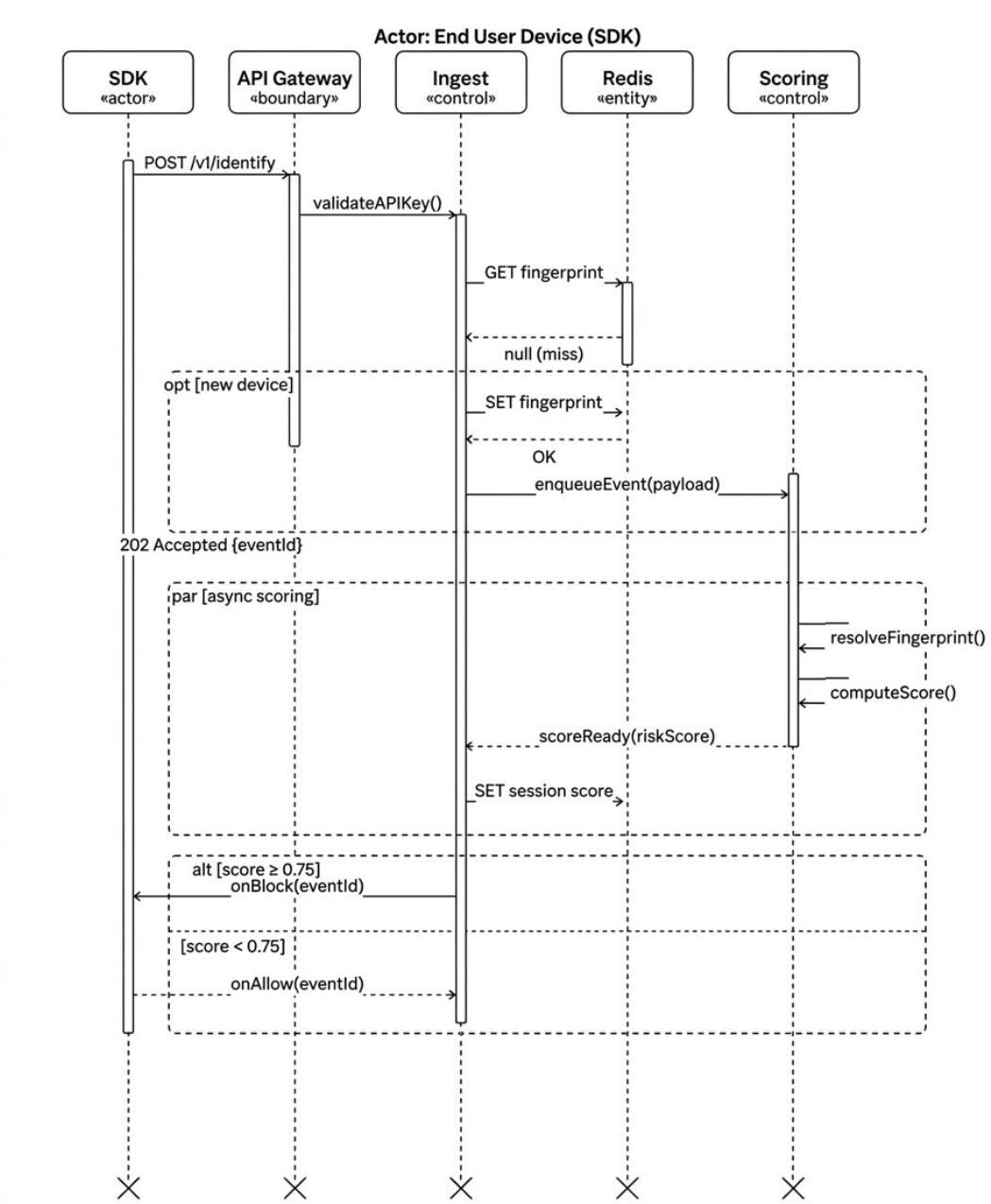

Figure 2: Sequence diagram 1: device identification (actor: End User Device / SDK).

Figure 2 shows the SDK identification path. The SDK sends `POST /v1/identify` to the API Gateway, which validates the API key through the Ingest service. Ingest looks up the fingerprint in Redis; on a cache miss it stores a new fingerprint record. Ingest enqueues an event to the Scoring service and immediately returns `202 Accepted` with an `eventId`. Scoring then resolves the fingerprint and computes a risk score asynchronously; Ingest updates the session score in Redis and notifies the SDK with `onBlock` or `onAllow` depending on the score threshold.

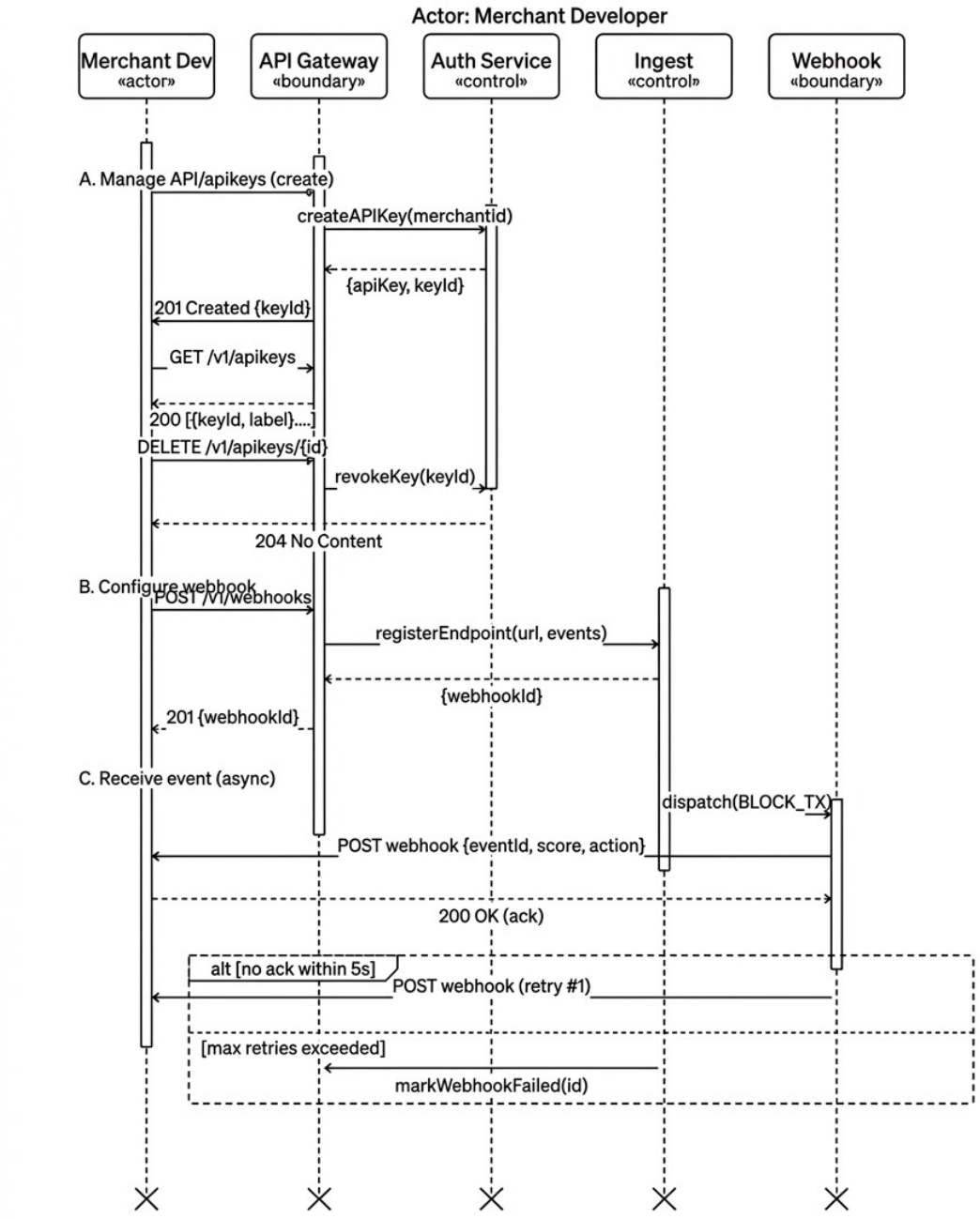

Figure 3: Sequence diagram 2: API keys and webhooks (actor: Merchant Developer).

Figure 3 covers merchant integration setup. In section A, the developer creates an API key (`POST`, `201 Created`), lists keys (`GET /v1/apikeys`), and revokes a key (`DELETE`, `204 No Content`) via the API Gateway and Auth Service. In section B, the developer registers a webhook endpoint (`POST /v1/webhooks`) through Ingest. In section C, when a transaction is blocked, Ingest dispatches a `BLOCK_TX` event to the Webhook service, which `POST`s the notification to the merchant; retries run if no acknowledgement arrives within five seconds, and failed deliveries are marked in Ingest after the maximum retry count.

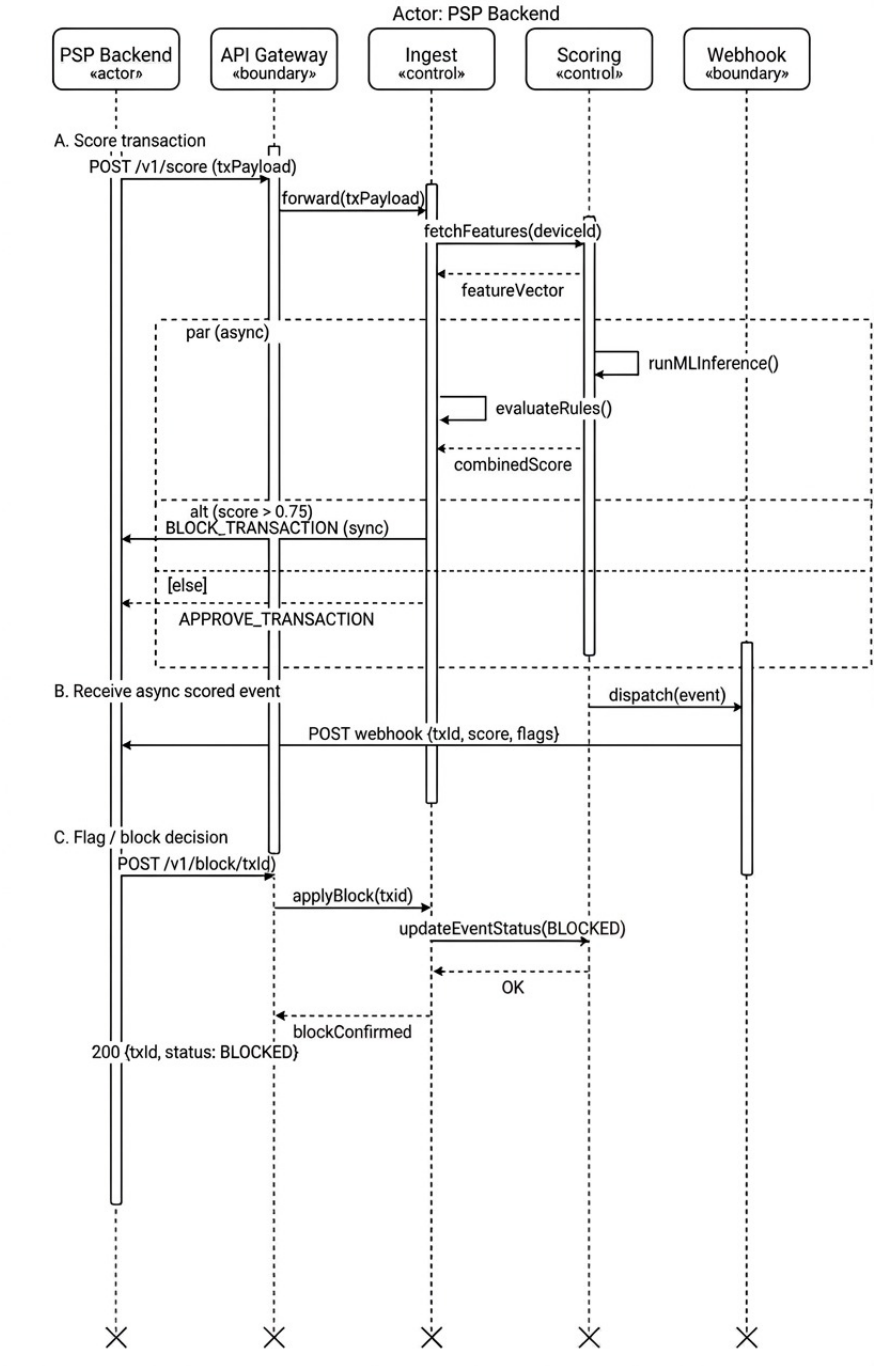

Figure 4: Sequence diagram 3: transaction scoring and blocking (actor: PSP Backend).

Figure 4 describes the PSP scoring path. In section A, the PSP Backend sends `POST /v1/score` with a transaction payload; the Gateway forwards it to Ingest, which fetches a feature vector from Scoring. Ingest evaluates rules to produce a combined score in parallel with background scoring work; if the score exceeds the threshold the PSP receives `BLOCK_TRANSACTION`, otherwise `APPROVE_TRANSACTION`. In section B, Scoring dispatches an asynchronous webhook event (`txId`, score, flags) back to the PSP. In section C, the PSP may confirm a block with `POST /v1/block/{txId}`, which updates the event status to `BLOCKED` and returns `200`.

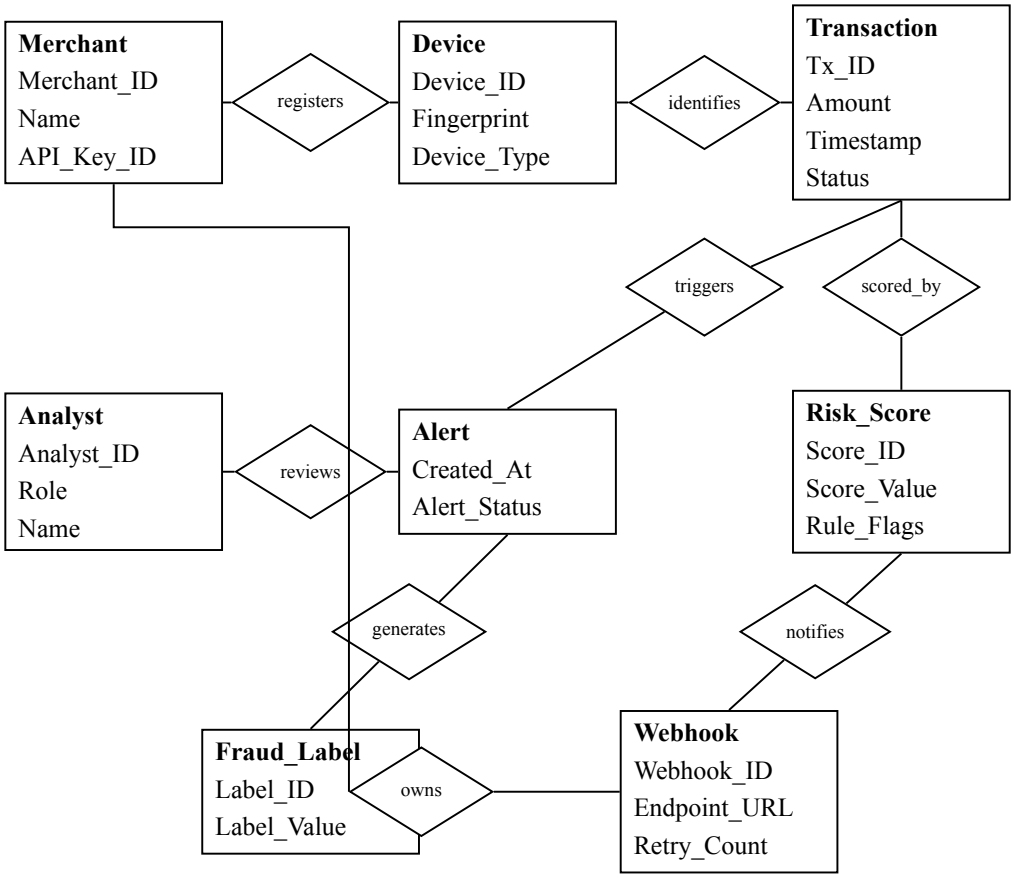

Figure 5: Entity-Relationship (ER) diagram of the planned database schema (rule-based scoring).

Figure 5 models the persistent data for the first release. A **Merchant** *registers* **Devices** and *owns* **Webhooks**. Each **Device** *identifies* **Transactions**. A transaction is *scored by* a **Risk_Score** record (rule flags and score value) and can *trigger* an **Alert**. An **Analyst** *reviews* alerts; confirmed cases *generate* **Fraud_Label** records for audit and trust updates. High-risk outcomes *notify* the merchant through the registered **Webhook** endpoint.

6 Proposed Performance Analysis Methodology and Validation Scheme

The team will check the system with simple, repeatable tests (load runs and automated scripts), not one-off demos only. Table 1 summarises what will be measured.

What we check	How	Target
Scoring speed	Short load test (k6)	Most requests under 50 ms
Traffic capacity	Increase load step by step	About 3,200 requests per second
Device lookup speed	API tests	Under 500 ms
Stability under load	Check error logs	Very few server errors
Overall behaviour	Automated test suite	At least 92% tests pass
SIM swap safety	Test a phone-number change	Block high-risk cases

Table 1: How the system will be tested (initial targets)

Demo at defenses will show ALLOW, FLAG, and BLOCK decisions on sample transactions.

7 Proposed Deliverable/Output

The planned deliverables are a **Go API**, a **web SDK**, supporting **documentation**, and background **workers** for processing and updates.

8 Project Task and Time Schedule

The team will follow the college semester and NCIT defense dates (proposal, mid-term, and final, with at least four weeks between defenses). The table below is only a rough plan; details will be updated with the supervisor in the meeting log.

Phase	Planned focus	Outcome
Proposal	Report, design, presentation	Proposal defense
Early semester	Setup and core development	Working prototype
Mid-term	Progress demo and report	Mid-term defense
Late semester	Testing and refinement	Stable demo
Final	Documentation and presentation	Final defense

Table 2: Indicative project schedule (initial proposal)

9 Bibliography/References

References

[1] Financial Intelligence Unit Nepal, *Annual Report 2023/24*, Nepal Rastra Bank, Kathmandu, 2024. Available at https://www.nrb.org.np/contents/uploads/2025/03/FIU-Nepal-Annual-Report-2080.81-202324.pdf (accessed Jun. 2026).

[2] P. Eckersley, How unique is your web browser?, In *Privacy Enhancing Technologies Symposium (PETS 2010)*, LNCS vol. 6205, pp. 1–18, Springer, Berlin, 2010. DOI: https://doi.org/10.1007/978-3-642-14527-8_1.

[3] P. Laperdrix, W. Rudametkin, and B. Baudry, Beauty and the beast: Diverting modern web browsers to build unique browser fingerprints, In *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 878–894, San Jose, CA, USA, 2016. DOI: https://doi.org/10.1109/SP.2016.57.

[4] A. Vastel, P. Laperdrix, W. Rudametkin, and R. Rouvoy, FP-STALKER: Tracking browser fingerprint evolutions, In *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 209–226, San Francisco, CA, USA, 2018. DOI: https://doi.org/10.1109/SP.2018.00008.

[5] R. J. Bolton and D. J. Hand, Statistical fraud detection: A review, *Statistical Science*, 17(3):235–255, 2002. DOI: https://doi.org/10.1214/ss/1042727940.

[6] J. West and M. Bhattacharya, Intelligent financial fraud detection: A comprehensive review, *Computers & Security*, 57:47–66, 2016. DOI: https://doi.org/10.1016/j.cose.2015.09.005.

[7] U. Fiore, A. De Santis, F. Perla, P. Zanetti, and F. Palmieri, Using generative adversarial networks for improving classification effectiveness in credit card fraud detection, *Information Sciences*, 479:448–455, 2019. DOI: https://doi.org/10.1016/j.ins.2019.03.045.

[8] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, Calibrating probability with undersampling for unbalanced classification, In *Proc. IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 2652–2659, Cape Town, South Africa, 2015. DOI: https://doi.org/10.1109/SSCI.2015.33.

[9] B. B. Mjaaland, J. Heeger, and S. D. Wolthusen, SIM swap: A new insider attack in the mobile banking ecosystem, In *Proc. IEEE Conference on Communications and Network Security (CNS)*, pp. 1–6, Avignon, France, 2021. DOI: https://doi.org/10.1109/CNS48642.2021.9470941.